

Relatório Técnico: Framework para Aplicações de Quiz

Disciplina: Princípios e Padrões de Projetos — BCC / FACOM / UFU

Docente: Profa. Fabíola S. F. Pereira

Alunos: Igor dos Reis Mascarenhas, Lavínia Oliveira Nascimento e Sanmyla Silva Costa

1. Arquitetura e Desacoplamento

Desenvolvemos o projeto dividindo a solução em dois módulos independentes: o núcleo reutilizável (**framework**) e as regras específicas das aplicações clientes (**aplicacao1** e **aplicacao2**).

- **Módulo Framework (**framework**)**: centraliza o controle do fluxo do jogo, a contagem e armazenamento de pontos e o gerenciamento das perguntas. Para manter a independência da plataforma, esse módulo não possui nenhuma dependência de bibliotecas de interface gráfica ou de entrada por terminal.
- **Módulos Clientes (**aplicacao1** e **aplicacao2**)**: consomem o framework sem alterar seu código-fonte. Eles fornecem o conjunto de perguntas, definem as regras de pontuação e constroem a interface de interação (uma versão com interface gráfica em Java Swing e outra via terminal de texto).

2. Aplicação dos Padrões de Projeto

A estrutura do projeto foi modelada aplicando os padrões recomendados:

- **Template Method (**JogoQuiz**)**: O método final **iniciar()** estabelece a sequência fixa da partida: percorrer a lista de perguntas, validar respostas, contabilizar erros/acertos e chamar a exibição do resultado. As etapas operacionais de renderização e captura de dados são declaradas como métodos abstratos (**exibirPergunta**, **capturarResposta** e **exibirResultadoFinal**), repassando a implementação para as subclasses.
- **Strategy (**EstrategiaPontuacao** & **GerenciadorPontuacao**)**: Isolou a lógica do cálculo de pontuação em uma interface funcional. Isso permitiu configurar sistemas de pontuação distintos para cada projeto (como \$+10/-2\$ na primeira aplicação e \$+10/-5\$ na segunda) sem precisar alterar a classe **GerenciadorPontuacao**.
- **Factory Method / Simple Factory (**FabricaPerguntas**)**: Organiza a instanciação de objetos do tipo **Pergunta**. A fábrica valida pré-requisitos antes de criar o objeto — como garantir a presença de pelo menos duas alternativas e a inclusão do gabarito correto —, evitando inconsistências em tempo de execução.
- **Observer (**QuizMultiplayer** & **JogadorObserver**)**: Implementação opcional para suporte a eventos e partidas multiplayer. As instâncias registradas recebem notificações automáticas sobre o envio de novas perguntas (**receberNovaPergunta**) e mudanças de placar (**atualizarPlacar**).

3. Reutilização e Resultados

A reutilização do código foi validada através do desenvolvimento de duas aplicações distintas sob o mesmo núcleo:

1. **Aplicação 1 (Quiz de Programação - Interface Swing)**: Extensão visual que utiliza janelas do tipo **JDialog**, botões **JRadioButton** e caixas de mensagem **JOptionPane**.
2. **Aplicação 2 (Quiz do Praia Clube - Console)**: Versão em modo texto que faz o tratamento de entradas do usuário com **Scanner** e exibe menus formatados no terminal.

Ambas as aplicações compartilham a mesma base de código do pacote **framework**, demonstrando o reuso das regras do jogo sem a necessidade de duplicar ou modificar o código original do framework.